

Building a Local Chatbot on a Raspberry Pi 5 (8 GB)

Ollama + Open WebUI: complete process, troubleshooting and maintenance

Overview

This guide follows the process in your PDF, from preparing the Raspberry Pi through to a working terminal chatbot (Steps 1 to 5), then the optional browser interface (Step 6), troubleshooting fixes, how to close or shut down, the optional API (Step 7) and maintenance commands.

Part	What it covers
Steps 1 to 5	OS update, Ollama install, model choice, download and terminal chat (already a working chatbot)
Step 6 (optional)	Open WebUI in Docker: a ChatGPT-style page in the browser
Troubleshooting	Checking logs and ports, tools error fix, login and password recovery
Closing and shutdown	Exit the terminal, stop stuck programs, shut down or reboot
Step 7 (optional)	Query Ollama from curl or Python through its local API
Maintenance	Useful everyday commands

Step 1: Use a 64-bit OS and update it

Use Raspberry Pi OS (64-bit), preferably Lite, to save RAM. Then run:

```
sudo apt update
sudo apt upgrade -y
sudo reboot
```

Check that you are on 64-bit after the reboot (the output should say `aarch64`):

```
uname -m
```

Step 2: Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

The article notes that the script sometimes crashes on the Pi. If that happens, use this version instead:

```
curl -fsSL --http1.1 https://ollama.com/install.sh | sh
```

Verify the install:

```
ollama --version
```

Ollama runs as a background service automatically. Ollama only uses the Pi's CPU, not its GPU, so expect modest speed.

Step 3: Pick a model for 8 GB RAM

The guide recommends models of about 4B parameters or less on a Pi 5. Good starting points:

Model	Command name	Notes
Gemma 3 1B	<code>gemma3:1b</code>	Fast, responsive
Qwen 0.5B	<code>qwen:0.5b</code>	Very fast, less accurate
Gemma 3 4B	<code>gemma3:4b</code>	Better answers, slower

The author tried 7B models on a Pi 5 8GB and found them accurate but too slow. Start with `gemma3:1b`, then move up to `gemma3:4b` if you want better quality.

Step 4: Download the model

```
ollama pull gemma3:1b
```

List what is installed:

```
ollama list
```

Step 5: Chat in the terminal

```
ollama run gemma3:1b
```

You will get a `>>>` prompt. Type your question and press Enter. Useful commands inside the chat:

- `/bye` exits the chat
- `/clear` resets the conversation context
- `/?` shows all available commands

To get speed stats (tokens per second) after each answer:

```
ollama run gemma3:1b --verbose
```

That is already a working chatbot. The next steps are optional.

Step 6 (optional): Add a browser chat interface with Open WebUI

This gives you a ChatGPT-style page you can open from your phone or PC. It runs in Docker.

6.1 Install Docker

```
sudo apt install docker.io -y
```

Make sure it starts now and on every boot:

```
sudo systemctl enable --now docker
```

Check it works:

```
sudo docker --version
sudo systemctl status docker
```

Press `q` to leave the status screen. You should see `active (running)`.

6.2 Make sure Ollama is running

Open WebUI needs Ollama's service up first:

```
curl http://localhost:11434
```

Expected reply: `Ollama is running`. If not, start it with `sudo systemctl start ollama`.

6.3 Start the Open WebUI container

The article runs the bundled `:ollama` image, which includes a second copy of Ollama inside the container. Since you already installed Ollama natively in Step 2, this cleaner variant connects to it instead and avoids downloading models twice:

```
sudo docker run -d --network=host \
-v open-webui:/app/backend/data \
-e OLLAMA_BASE_URL=http://127.0.0.1:11434 \
-e PORT=3000 \
--name open-webui --restart always \
ghcr.io/open-webui/open-webui:main
```

What each part does:

Part	Meaning
<code>-d</code>	Runs in the background
<code>--network=host</code>	The container shares the Pi's network, so it can reach Ollama at <code>127.0.0.1:11434</code>
<code>-v open-webui:/app/backend/data</code>	A Docker volume that keeps your accounts and chat history if the container is recreated
<code>-e OLLAMA_BASE_URL=...</code>	Tells WebUI where Ollama lives
<code>-e PORT=3000</code>	The web page will be on port 3000
<code>--restart always</code>	Starts automatically after a reboot
<code>--name open-webui</code>	Container name, used for the management commands below

If the terminal is stuck showing output, press `Ctrl + C`. You will get your normal prompt back (for example `smfahadpi5@SMFahadPi5:~ $`).

Checking that the container is running and ready

List all containers:

```
sudo docker ps -a
```

1. Read the logs:

```
sudo docker logs --tail 40 open-webui
```

What to do:

1. Wait a few minutes, then check whether it has finished:

```
sudo docker logs --tail 15 open-webui
```

You are looking for a line saying Uvicorn is running, or an Open WebUI banner (large ASCII text). Once you see that, it is ready.

2. Check that the port is now listening:

```
sudo ss -tlnp | grep -E '3000|8080'
```

If you see `:3000`, open `http://localhost:3000` in Chromium. If you see `:8080`, use `http://localhost:8080` instead.

Example of a healthy result in your screenshot: the `ss` command showed `LISTEN` on `0.0.0.0:3000` owned by `python3`, and `sudo docker logs --tail 30 open-webui` showed `uvicorn` log lines with `GET /api/v1/auths/`, `/api/models`, `/api/v1/tools/` and similar requests returning `200`.

The first run downloads a large image (a few GB), so it can take several minutes. Watch progress with:

```
sudo docker logs -f open-webui
```

Press `Ctrl+C` to stop watching. The container keeps running. WebUI is ready when the logs stop scrolling and show it listening.

6.4 Open it in a browser

Get your Pi's IP:

```
hostname -I
```

On the Pi itself, go to `http://localhost:3000`. From another device on the same network, go to `http://<pi-ip>:3000`, for example `http://192.168.1.100:3000`. Use `http`, not `https`.

Startup can take 1 to 2 minutes after the container launches, so refresh if it does not load at first.

Open the page, create your account and start chatting

In Chromium, go to `http://localhost:3000` and click Reload. Then follow Step 6.5:

1. Click Get Started.
2. Create the admin account (name, email, password).
3. Pick your model (`gemma3:1b`) from the dropdown at the top.
4. Type a message and press Enter.

In more detail:

Create your account:

1. Click Get Started.
2. Enter a name, email and password, then click Create Admin Account. The email does not need to be real, since the account is stored only on your Pi.

Start chatting:

1. Click the model dropdown at the top of the screen.
2. Select `gemma3:1b` (or whichever model you pulled).
3. Type a message and press Enter.

The first reply takes longer because Ollama loads the model into RAM. After that it is faster.

Troubleshooting

Fix 1: Turn off native function calling (most likely cause of the tools error)

1. In the chat, click the controls icon in the top right (the sliders icon).
2. Open Advanced Params.
3. Find Function Calling and set it to Default (not Native).
4. Send your message again.

To make this permanent for the model:

Click your profile icon (bottom left), then Admin Panel, then Settings, then Models.

Click the pencil/edit icon next to `gemma3:1b`.

Open Advanced Params, set Function Calling to Default, and click Save.

Untick these in Capabilities

In the model's edit page, the Capabilities section lists: Vision, File Upload, File Context, Web Search, Image Generation, Code Interpreter, Terminal, Usage, Citations, Status Updates, Memory and Builtin Tools. Under Default Features there are Web Search and Code Interpreter. In your screenshot, Vision, File Upload, File Context, Web Search, Code Interpreter and Builtin Tools were ticked. Untick the following:

- Builtin Tools (the most important one). Once it is off, the long Builtin Tools list in your second screenshot should disappear or stop applying, so you do not need to untick each item.
- Web Search
- Code Interpreter
- Vision, because `gemma3:1b` is text-only

You can leave File Upload and File Context ticked. They only matter if you upload documents, and they do not cause the tools error.

Leave Default Features (Web Search, Code Interpreter) unticked, as they already are.

If the email and password are incorrect (cannot sign in)

If the Sign in to Open WebUI page rejects your email and password, you can reset the password from the Pi. First list the emails stored in the database to confirm the exact one:

```
sudo docker exec open-webui python3 -c "import sqlite3;
c=sqlite3.connect('/app/backend/data/webui.db'); print(c.execute('select
email from auth').fetchall())"
```

Then set a new password for that email (replace `NEW_PASSWORD` and `YOUR_EMAIL` with your own values; the screenshot used the account's real email and password, which are left out here):

```
sudo docker exec open-webui python3 -c "import sqlite3,bcrypt;
h=bcrypt.hashpw(b'NEW_PASSWORD',bcrypt.gensalt()).decode();"
```

```
c=sqlite3.connect('/app/backend/data/webui.db'); c.execute('update auth set password=? where email=?', (h, 'YOUR_EMAIL')); c.commit(); print('done')
```

The terminal prints `done` when it has worked. Then sign in again with the new password.

Closing the terminal and shutting down

Close the terminal window

Type this and press Enter:

```
exit
```

Or press Ctrl + D. Either one closes the terminal window (or ends the SSH session, if you connected that way). You can also click the X at the top right of the window.

Exit a running program (the terminal is stuck)

- Ctrl + C stops a running command, such as a stuck `docker logs -f open-webui`.
- q closes screens like `systemctl status` and `less`.
- /bye exits the Ollama chat if you are inside `ollama run gemma3:1b`.
- Ctrl + D also exits the Ollama chat.

Shut down or reboot the Pi

Only do this when you want to power it off:

```
sudo shutdown now
```

Wait until the green light stops flashing before you unplug the power. To reboot instead:

```
sudo reboot
```

If you are using Raspberry Pi Connect, shutting down will disconnect your session, and you will need to power the Pi back on physically to reconnect.

Step 7 (optional): Query it from Python or curl

Ollama also exposes a local API on port 11434, which is useful if you want to build the chatbot into your own program:

```
curl http://localhost:11434/api/chat -d '{
  "model": "gemma3:1b",
  "messages": [{"role": "user", "content": "Hello!"}],
  "stream": false
}'
```

Useful maintenance commands

```
ollama ps          # show models currently loaded in RAM
ollama rm gemma3:1b # delete a model to free disk space
sudo systemctl status ollama # check the Ollama service
```

```
sudo docker ps          # check Open WebUI is running
vcgencmd measure_temp    # check the Pi's temperature
```